Proyecto final

Rutas de turismo sostenible en Santa Marta

Una solución web semántica para recomendar rutas según el perfil del turista y el estado real de cada zona

Integrantes
Doris Arzuaga
Gabriela Zabaleta

Curso: Ingeniería Web
Fecha: Octubre de 2026

1. Formulación del problema

1.1 Descripción del problema

Quien quiere conocer la Sierra Nevada y la costa de Santa Marta se encuentra con una oferta amplia, pero repartida en blogs, redes sociales, agencias y páginas oficiales. Cada fuente cuenta una parte: una dice cuánto dura la caminata, otra cuánto cuesta, y casi ninguna dice si la zona está abierta ese día. Esto último importa mucho. El Parque Tayrona tiene cierres periódicos acordados con los pueblos indígenas, tuvo un cierre por seguridad en febrero de 2026 y maneja cupos diarios por sector. Ciudad Perdida, además, solo se puede visitar con una agencia autorizada, porque es un sitio sagrado para el pueblo Kogui.

El resultado es que el turista decide con información incompleta o vencida. Puede planear una ruta hacia una zona cerrada, escoger un trek que no corresponde a su condición física o llegar a un lugar que ya alcanzó su capacidad. Cuando muchos visitantes se concentran en los mismos sitios, el impacto sobre el ecosistema y sobre las comunidades crece. Las autoridades y los operadores, por su parte, no tienen un canal único para publicar cierres, cupos y normas.

1.2 Usuarios involucrados

Usuario	Quién es	Qué necesita
Turista	Persona local, nacional o extranjera que quiere visitar la zona	Saber qué ruta le conviene según su perfil y su fecha, y si esa ruta está disponible
Guía	Guía local certificado	Publicar los recorridos que ofrece y recibir solicitudes
Operador turístico	Agencia autorizada, por ejemplo para Ciudad Perdida	Ofrecer rutas con cupos y administrar sus reservas
Autoridad ambiental	Funcionario de Parques Nacionales o de la entidad de turismo del distrito	Publicar cierres, topes de capacidad y normas de conducta

1.3 Objetivo de la solución

Diseñar una plataforma web que recomiende rutas de turismo sostenible en Santa Marta a partir del perfil del turista, la fecha y el estado real de cada zona, y que integre en un mismo lugar a quienes ofrecen las rutas y a quienes las regulan.

Objetivos específicos:

- Representar con una ontología las rutas, las zonas, las normas y los perfiles, para que las reglas queden explícitas y no escondidas en el código.
- Recomendar rutas compatibles con el perfil del turista y descartar las que no se pueden hacer por cierre, cupo lleno o falta de operador autorizado.
- Permitir que la autoridad ambiental publique cierres y límites y que estos afecten de inmediato las recomendaciones.
- Promover rutas de menor impacto cuando una zona sensible esté cerca de su límite.

1.4 Alcance

Incluye: el diseño del problema, la arquitectura, la metodología, los modelos conceptual y navegacional, un prototipo de las interfaces principales, el diseño de los servicios web, la ontología en Protégé y el grafo de conocimiento en Neo4j con sus consultas.

No incluye:

- Una aplicación funcional en producción, porque el proyecto no la exige.
- Pagos en línea ni reservas reales con operadores.
- Datos reales de personas. Trabajamos con información pública, ficticia o sintética. Las cifras de capacidad de carga son ilustrativas, porque las fuentes consultadas no coinciden entre sí.
- Cobertura de toda la región. Nos limitamos a un conjunto de rutas representativas de Santa Marta y la Sierra Nevada.

1.5 Funcionalidades principales

1. **Crear el perfil del turista:** intereses, condición física, días disponibles y presupuesto.
2. **Recomendar rutas** según el perfil y la fecha de visita.
3. **Consultar la disponibilidad de una ruta:** si la zona está abierta, si hay cupo y qué normas aplican.
4. **Solicitar una ruta guiada** a un guía u operador autorizado.
5. **Gestionar la oferta de rutas:** el guía o el operador publica sus recorridos, duración, dificultad y cupos.
6. **Publicar cierres, límites de capacidad y normas:** lo hace la autoridad ambiental.

1.6 Por qué el problema necesita una solución web

- **La información cambia y todos necesitan la misma versión.** Un cierre decidido hoy tiene que verse hoy en las recomendaciones. Un documento o un folleto impreso no lo permite.
- **Hay varios actores con funciones distintas** que deben compartir datos: quien viaja, quien ofrece, quien regula.
- **Se consulta antes y durante el viaje, desde el celular.** Un navegador no exige instalar nada, y eso incluye al turista extranjero que apenas llega.
- **Las rutas dependen de reglas que se combinan** (fecha, cierre, cupo, dificultad, operador). Esto es más que consultar y guardar datos: es un problema de significado, y por eso se justifica una ontología y un grafo de conocimiento.

2. Fundamentación

No quisimos resumir cada fuente por separado. Preferimos usarlas para justificar decisiones concretas del proyecto, y eso es lo que hacemos a continuación.

Metodología. Elegimos UWE porque propone modelar una aplicación web en capas distintas, con una notación basada en UML y sus extensiones: requisitos, modelo conceptual, modelo navegacional y modelo de presentación (Koch et al., 2008). Su base en UML también permite expresar la navegación con diagramas conocidos, algo que Koch y Kraus (2002) estudian en detalle. Esa separación nos sirvió para trabajar el dominio y la navegación por aparte, y para que cada fase terminara en un producto que se pudiera revisar.

Arquitectura y servicios. Diseñamos los servicios como recursos REST porque es un estilo pensado para sistemas distribuidos en la web, con una interfaz uniforme, comunicación sin estado y recursos identificados por URL (Fielding, 2000). Esas propiedades son las que nos permiten sostener que la API puede crecer añadiendo instancias, y que el cliente puede ser una página web sin instalar nada.

Ontología. Seguimos el orden que recomiendan Noy y McGuinness (2001): primero definir el dominio y las preguntas de competencia, luego listar los términos importantes, después las clases y sus propiedades, y por último los individuos. Por eso las tres preguntas de competencia aparecen antes que el código de la ontología, y por eso separamos con cuidado las clases (por ejemplo `Zona`) de los individuos (por ejemplo `ZonaTayrona`).

Grafo de conocimiento. Usamos un grafo porque el problema es, en esencia, un conjunto de relaciones entre rutas, zonas, cierres y personas. Hogan et al. (2021) presentan los grafos de conocimiento como datos organizados en nodos y relaciones, a los que se les agrega un esquema o una ontología para dar significado y poder razonar sobre ellos. Eso corresponde a lo que hicimos: la ontología define el significado y el grafo responde las consultas.

Usabilidad. Evaluamos el prototipo con las heurísticas de Nielsen (2020), que sirven para detectar problemas de una interfaz antes de probarla con usuarios. Como recuerda la propia fuente, no reemplazan una prueba de uso: encuentran problemas de otro tipo. Por eso dejamos esa prueba como siguiente paso.

3. Arquitectura web

3.1 Arquitectura seleccionada

Elegimos una **arquitectura web en capas con servicios REST y una capa semántica**. El cliente es una aplicación web de una sola página que consume una API; la API contiene la lógica de recomendación y consulta dos almacenes: una base relacional para los datos operativos y un grafo de conocimiento para las reglas y relaciones del dominio.

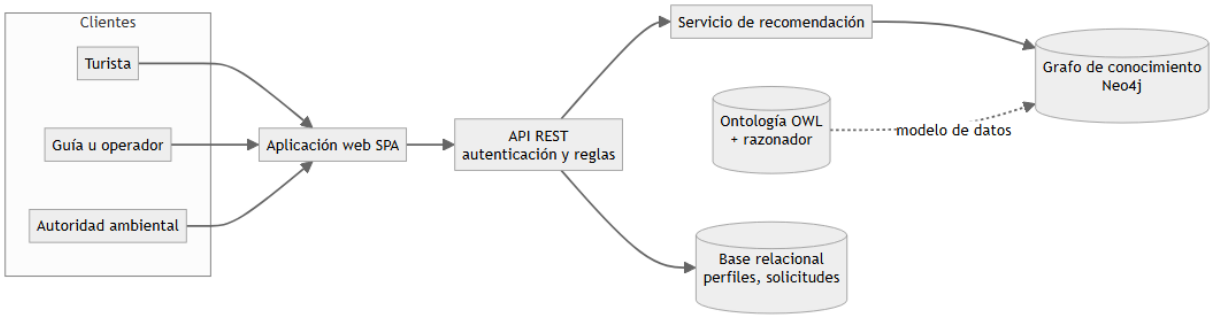


Figura 1. Arquitectura de la solución

3.2 Responsabilidad de cada componente

Componente	Responsabilidad
Aplicación web (SPA)	Mostrar las interfaces de cada perfil y consumir la API. No contiene reglas de negocio.
API REST	Punto único de entrada. Valida la identidad y los permisos y coordina los demás componentes.
Servicio de recomendación	Recibe perfil y fecha, consulta el grafo y devuelve las rutas aptas y las descartadas con su motivo.
Base relacional	Guarda lo que cambia por usuario: perfiles, solicitudes de ruta, cuentas.
Grafo de conocimiento (Neo4j)	Guarda rutas, zonas, cierres, normas y sus relaciones, y responde las consultas de recomendación.
Ontología (OWL)	Define el vocabulario y las reglas del dominio. Valida con un razonador que el modelo es consistente. El grafo se carga con esa estructura.

3.3 Cómo favorece los atributos de calidad

- **Mantenibilidad:** las reglas (qué es una zona cerrada, qué ruta exige guía) viven en la ontología y el grafo, no repartidas en el código. Si cambia una norma se edita el dato, no el programa.
- **Seguridad:** el cliente nunca consulta los almacenes directamente. Solo la API los toca, y allí se aplican los permisos por rol: únicamente la autoridad ambiental publica cierres y únicamente un operador autorizado publica rutas de trekking.
- **Escalabilidad:** la API no guarda estado, así que se pueden agregar instancias. La consulta de cierres y cupos es de solo lectura y se puede almacenar en caché con una vigencia corta.
- **Usabilidad:** la recomendación explica por qué una ruta se descartó ("zona cerrada del 1 al 15 de febrero"), en lugar de ocultarla. Esto reduce la frustración del turista.

4. Metodología de Ingeniería Web

4.1 Metodología seleccionada: UWE

Aplicamos **UWE (UML-based Web Engineering)**, porque separa de forma explícita el análisis de requisitos, el modelo conceptual, el modelo navegacional y el modelo de presentación. Eso coincide con lo que se pide entregar en el proyecto.

4.2 Fases, actividades y productos

Fase	Actividades realizadas	Productos
Requisitos	Identificar los cuatro perfiles, sus necesidades y las seis funcionalidades	Documento de formulación (sección 1) y casos de uso
Modelo conceptual	Definir las clases del dominio y sus relaciones	Diagrama conceptual y ontología
Modelo navegacional	Definir cómo se mueve cada perfil entre las pantallas	Diagrama de navegación
Modelo de presentación	Diseñar las interfaces y evaluarlas con heurísticas	Prototipo y evaluación de usabilidad
Arquitectura y servicios	Elegir la arquitectura y diseñar los servicios	Diagrama de arquitectura y catálogo de servicios

4.3 Relación con las necesidades del usuario

Cada fase parte de lo que necesita un perfil. Por ejemplo, la necesidad del turista ("saber si puede ir") determina un caso de uso (consultar disponibilidad), que exige las clases Zona y Cierre en el modelo conceptual, una pantalla de detalle de ruta en el modelo navegacional y una consulta Cypher en el grafo.

4.4 Ventajas y limitaciones para este proyecto

- **Ventajas:** los modelos están separados, por lo que el dominio (ontología) y la navegación se pueden revisar de forma independiente; usa UML, que ya conocemos; y cada modelo se relaciona directamente con un entregable.
- **Limitaciones:** es una metodología de diseño que asume buena parte de los requisitos desde el inicio, así que no refleja bien los cambios frecuentes. Además, la documentación de los modelos puede crecer más de lo necesario para un proyecto de este tamaño.

5. Modelado y usabilidad

5.1 Casos de uso por perfil

Perfil	Casos de uso
Turista	Crear perfil · Pedir recomendación · Consultar disponibilidad · Solicitar ruta guiada
Guía	Publicar recorrido · Atender solicitudes
Operador	Publicar rutas y cupos · Gestionar solicitudes
Autoridad ambiental	Publicar cierres · Definir límites de capacidad · Publicar normas

5.2 Modelo conceptual

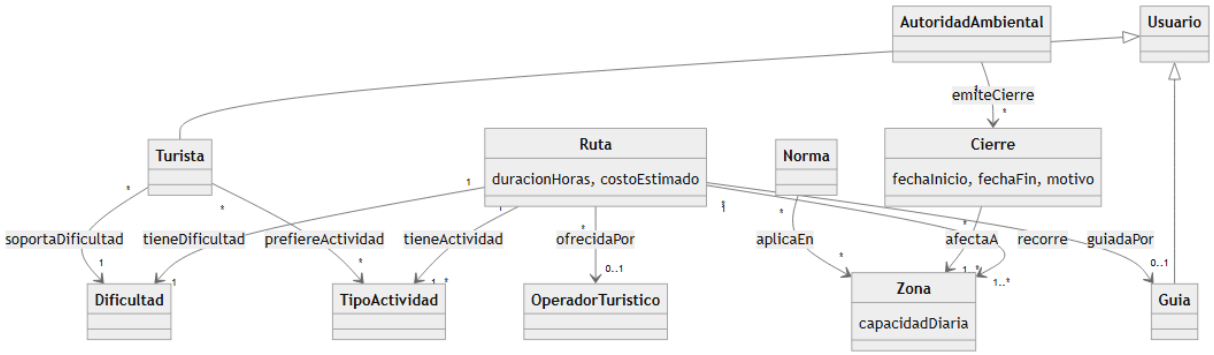


Figura 2. Modelo conceptual del dominio

5.3 Modelo navegacional

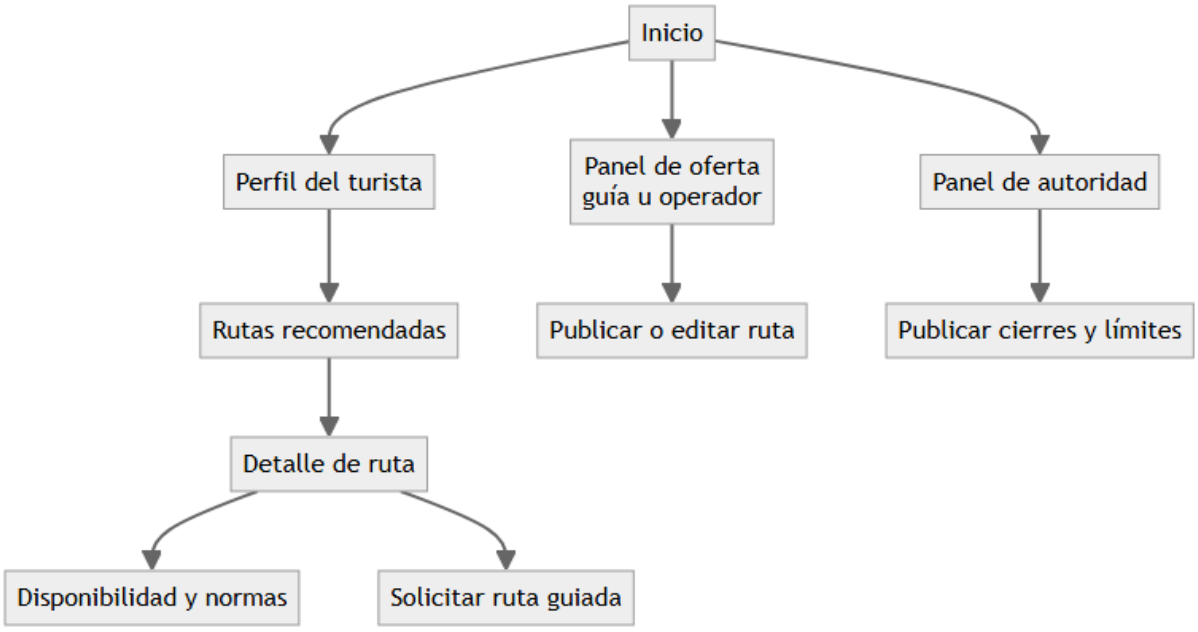


Figura 3. Modelo navegacional

5.4 Prototipo de las interfaces principales

Construimos un prototipo navegable en HTML, CSS y JavaScript (prototipo/index.html), que se abre en cualquier navegador y funciona sin servidor. Su código está separado en módulos que siguen la arquitectura de la sección 3: datos.js hace las veces del grafo, servicios.js simula la API REST con un método por cada servicio de la sección 6 y contiene las reglas del dominio, cada pantalla tiene su propio módulo en js/vistas/ y router.js implementa el modelo navegacional. Las vistas solo consultan los servicios, nunca los datos directamente, igual que la SPA solo habla con la API. Tiene cuatro pantallas: el perfil del turista, las rutas recomendadas, el detalle de una ruta y los paneles de oferta y de la autoridad ambiental. Las reglas son las mismas de la ontología: un cierre vigente descarta la ruta, y la dificultad y la actividad se comparan con el perfil. Los datos son ilustrativos.

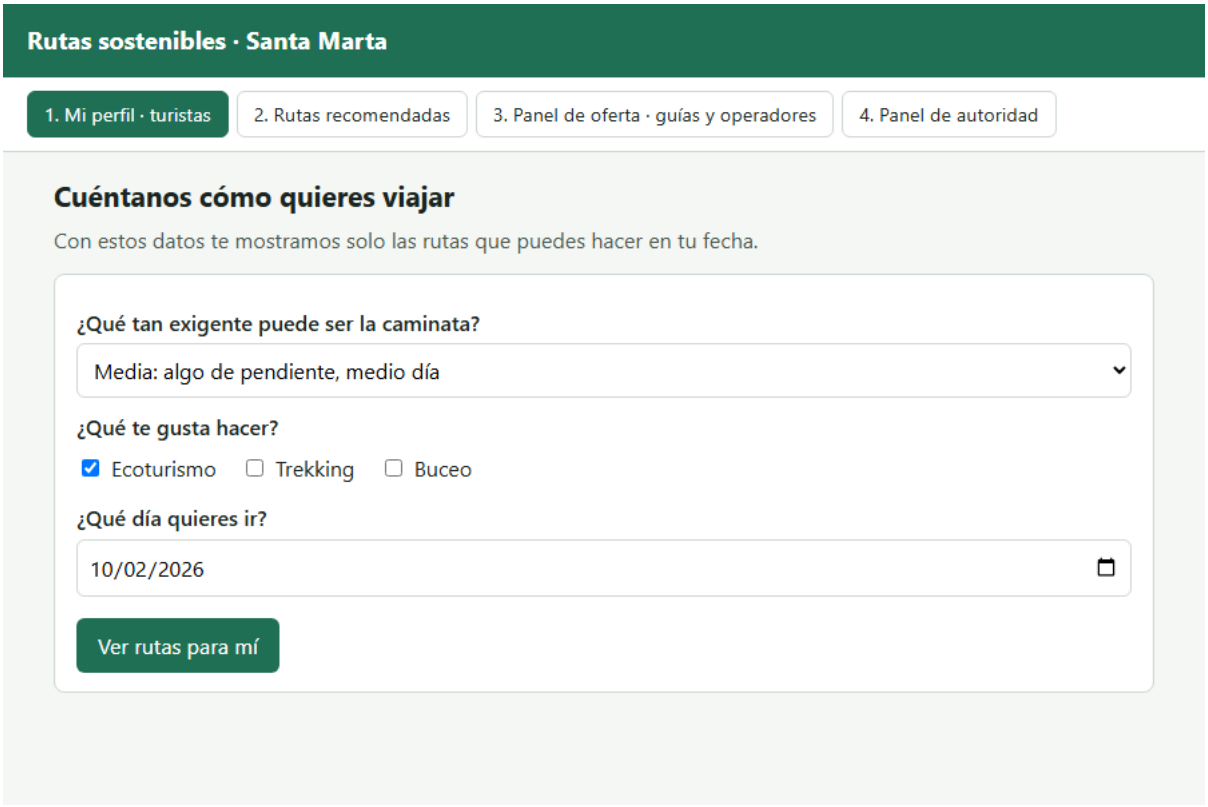


Figura 4. Pantalla de perfil del turista

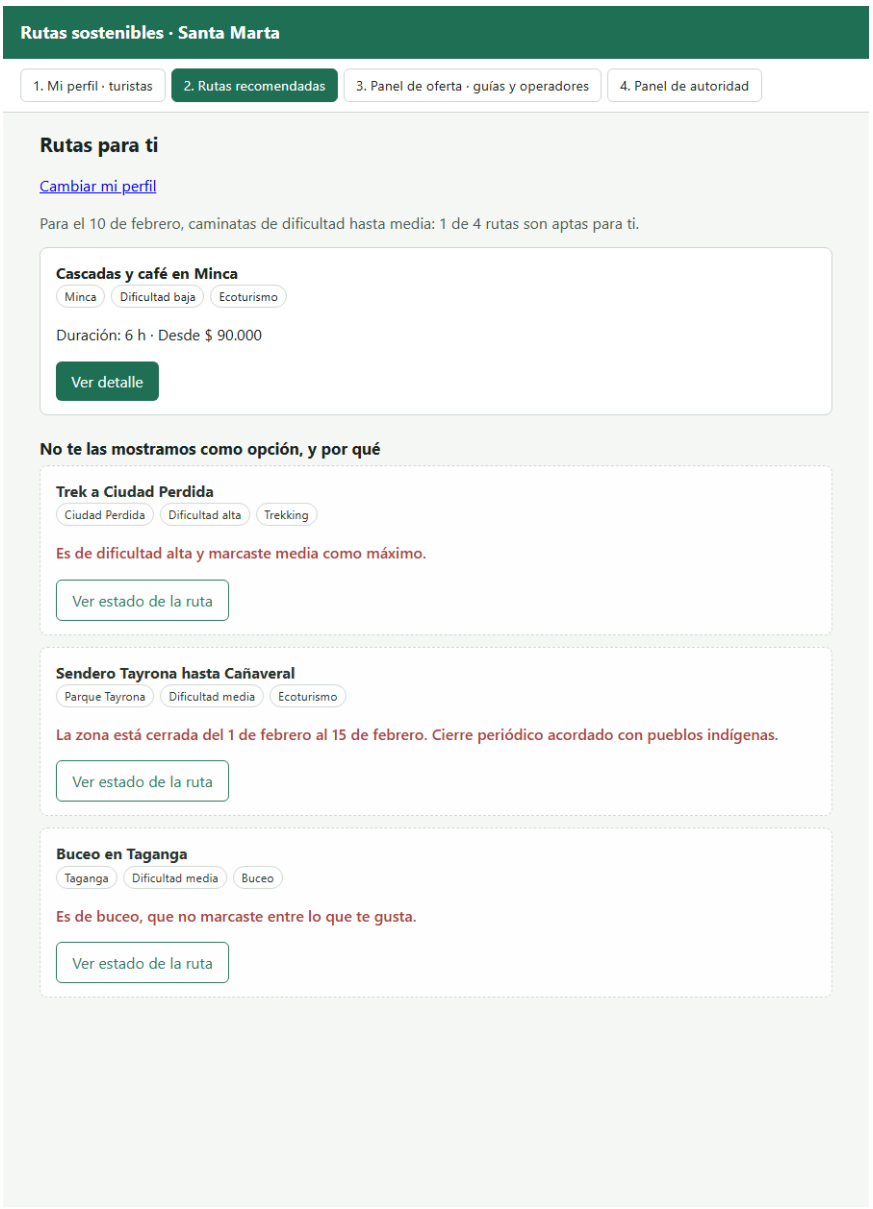


Figura 5. Rutas recomendadas para una turista que soporta dificultad media y prefiere ecoturismo, el 10 de febrero



Figura 6. Detalle de una ruta cuya zona está cerrada en la fecha elegida

5.5 Evaluación con heurísticas de Nielsen

Revisamos la primera versión del prototipo con cinco de las diez heurísticas de Nielsen (2020). Para cada una anotamos lo que encontramos y lo que cambiamos en la segunda versión.

Heurística	Qué encontramos en la versión 1	Qué mejoramos
1. Visibilidad del estado del sistema	Las rutas descartadas no se podían abrir, y el botón "Solicitar" de una ruta con zona cerrada no explicaba por qué no servía.	Las rutas descartadas tienen "Ver estado de la ruta". La ficha muestra si la zona está abierta o cerrada en la fecha elegida, con un aviso que da el motivo, y el botón desactivado se ve desactivado.
2. Correspondencia con el mundo real	El resumen decía "dificultad máxima media", un lenguaje técnico para una persona que viaja.	Ahora dice "caminatas de dificultad hasta media".
3. Control y libertad del usuario	Desde los resultados no había un camino directo para corregir el perfil. Además, se podía abrir "Detalle de ruta" desde el menú sin haber elegido ninguna, y mostraba una ruta cualquiera.	Se agregó el enlace "Cambiar mi perfil". El detalle sin ruta elegida regresa a la lista, y salió del menú.
4. Consistencia y estándares	El menú numeraba cinco pantallas de tres perfiles distintos sin decir para quién era cada una.	El menú indica el perfil: "turistas", "guías y operadores" y "autoridad".
5. Prevención de errores	El panel de la autoridad aceptaba un cierre cuya fecha final era anterior a la inicial.	Se valida y se muestra un mensaje que pide corregir las fechas antes de publicar.

Todavía no hicimos pruebas con personas usuarias. Una evaluación heurística encuentra problemas distintos a los de una prueba de uso, y es el paso siguiente natural.

6. Servicios web

Servicio	Propósito	Entrada	Salida	Método	Lo usa
`/turistas`	Crear el perfil	nombre, dificultad que soporta, actividades preferidas	id del perfil	POST	Turista
`/rutas/recomendadas`	Recomendar rutas	id de perfil, fecha	rutas aptas y rutas descartadas con motivo	GET	Turista (SPA)
`/rutas/{id}/disponibilidad`	Estado de una ruta en una fecha	id de ruta, fecha	abierta o cerrada, cupo, normas	GET	Turista
`/solicitudes`	Pedir una ruta guiada	id de ruta, id de perfil, fecha	id y estado de la solicitud	POST	Turista
`/rutas`	Publicar una ruta	nombre, zonas, dificultad, actividades, duración, costo	ruta creada	POST	Guía, operador
`/zonas/{id}/cierres`	Publicar un cierre	fechas, motivo	cierre creado	POST	Autoridad ambiental

El diseño es REST: los recursos son sustantivos (`rutas`, `zonas`, `solicitudes`) y los métodos HTTP expresan la acción. La respuesta de `/rutas/recomendadas` incluye las rutas descartadas y su razón, porque es lo que usa la interfaz para explicarse.

6.1 Un servicio expresado en SOAP/WSDL

`ConsultarDisponibilidad` equivale a `GET /rutas/{id}/disponibilidad`. En SOAP se describiría con un WSDL que declara una operación con un mensaje de entrada (`idRuta`, `fecha`) y un mensaje de salida (`estado`, `cupoRestante`). Se llamaría con un POST único a un endpoint y el contrato se leería del WSDL. En REST el contrato lo define la URL y el método. SOAP daría más formalidad al contrato; REST es más sencillo de consumir desde una SPA.

7. Ontología del dominio

Archivos: `rutas_sostenibles.owl` y su copia en Turtle. Se genera con `construir_ontologia.py` y se verifica con `verificar_ontologia.py`.

Requisito	Cantidad pedida	Lo que tiene
Clases	8	15 (con las definidas)
Propiedades de objeto	10	12
Propiedades de datos	5	7
Restricciones	3	4 (más las 4 clases definidas)
Individuos	8	21

Dominios y rangos están definidos en todas las propiedades de objeto.

Preguntas de competencia

1. ¿Qué rutas no están disponibles porque recorren una zona con cierre vigente?
2. ¿Qué rutas son de alta dificultad y exigen guía?
3. ¿Qué rutas son aptas para un turista que soporta dificultad media y prefiere ecoturismo?

Razonador y consistencia. Se ejecutó HermiT sobre la ontología: no hay clases inconsistentes.

Inferencias explicadas

1. `ZonaTayrona` queda clasificada como **ZonaCerrada**, porque tiene un cierre vigente (`tieneCierreVigente some Cierre`). Nadie la marcó así. A partir de eso, `RutaTayronaCanaveral` queda como **RutaNoDisponible**, porque recorre una zona cerrada.
2. `RutaCiudadPerdida` queda clasificada como **RutaAltaDificultad** y **RutaGuiadaObligatoria**, a partir de sus relaciones con la dificultad Alta y la actividad Trekking.

Limitación. El razonador no compara la fecha de consulta con las fechas del cierre. Por eso la relación `tieneCierreVigente` la afirma el servicio cuando la fecha cae dentro del cierre. La comparación de fechas se hace en la consulta del grafo.

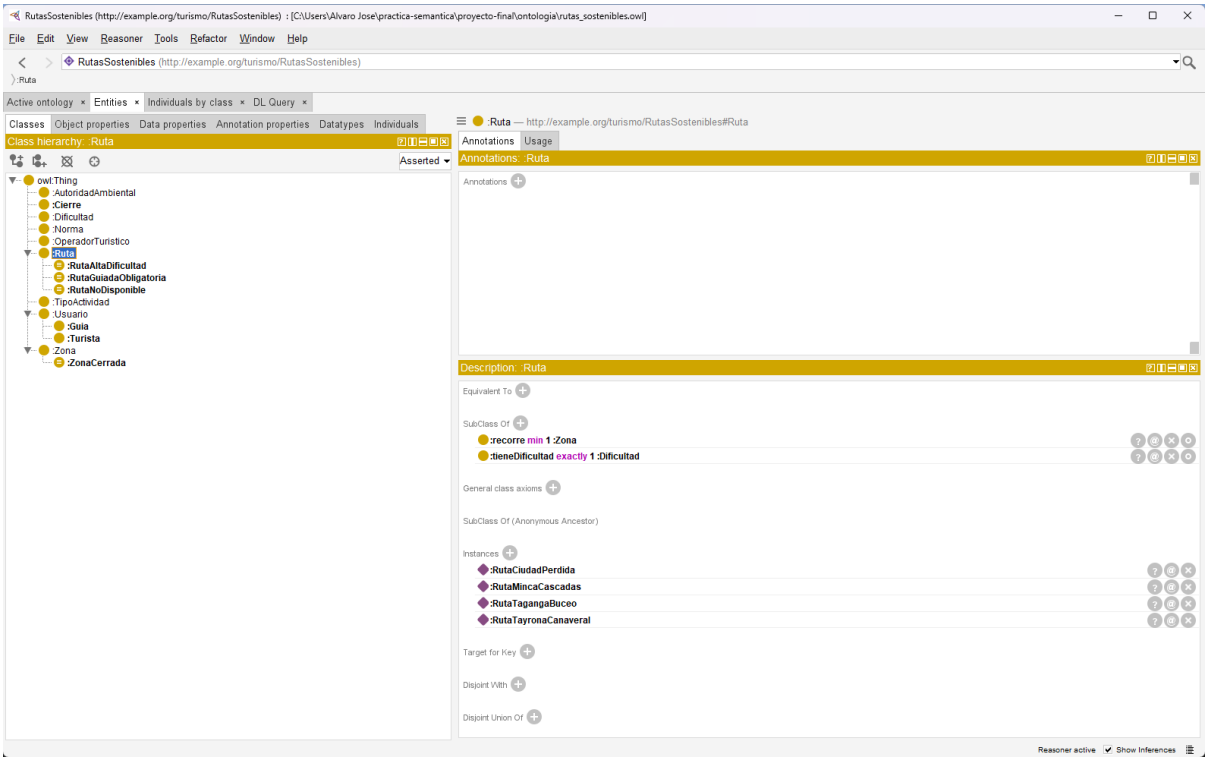


Figura 7. Jerarquía de clases en Protégé, con la clase Ruta, sus restricciones y sus instancias

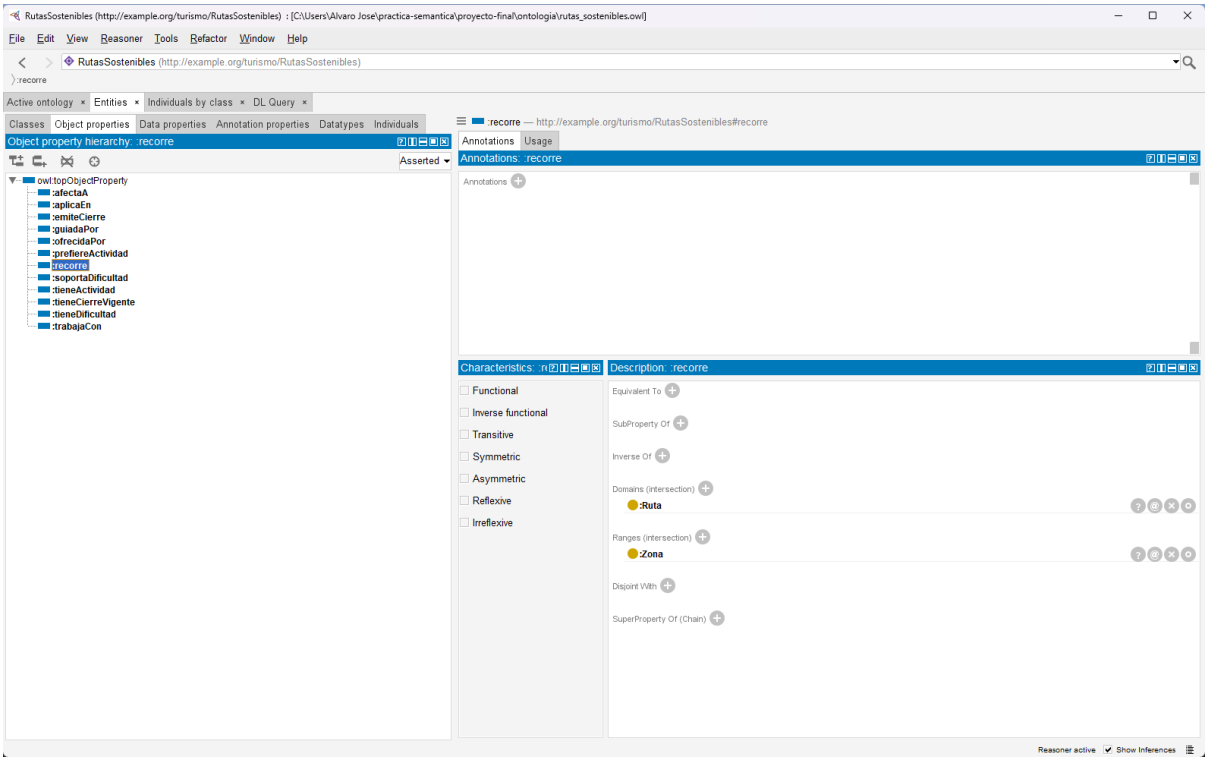


Figura 8. Propiedad de objeto recorre, con dominio Ruta y rango Zona

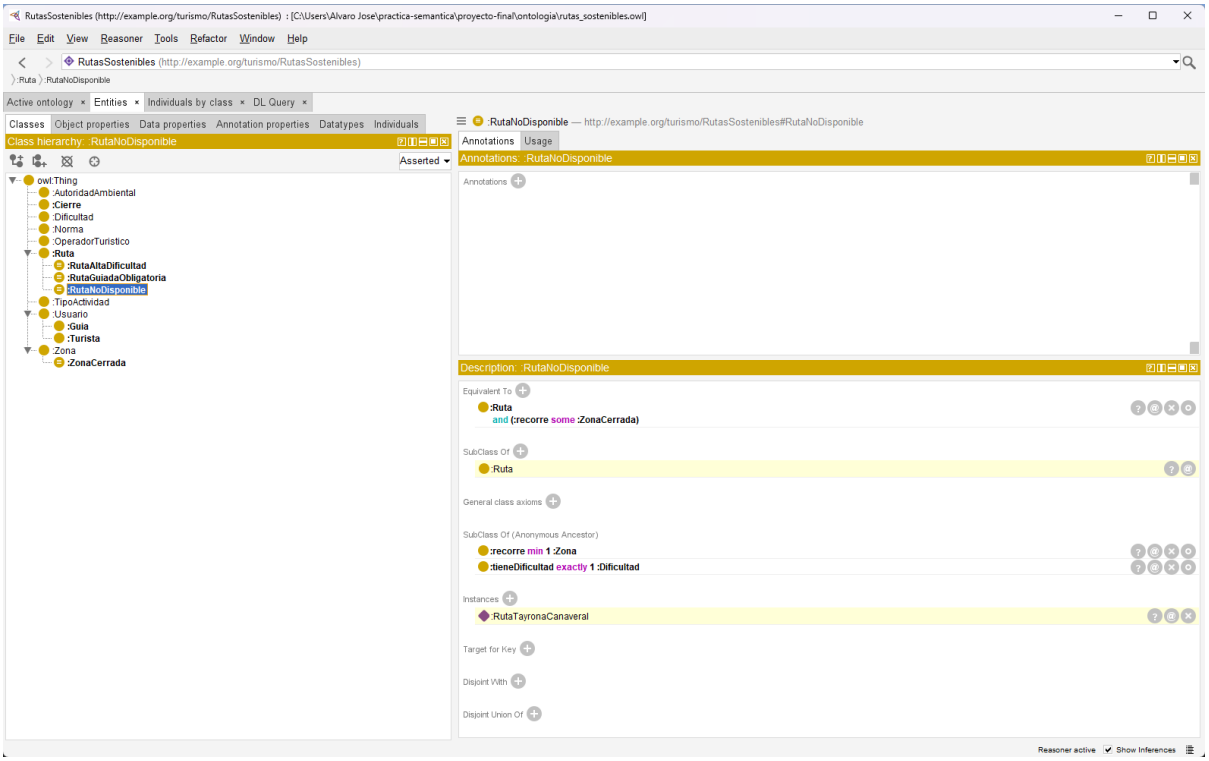


Figura 9. Clase definida RutaNoDisponible y la instancia que el razonador le asignó

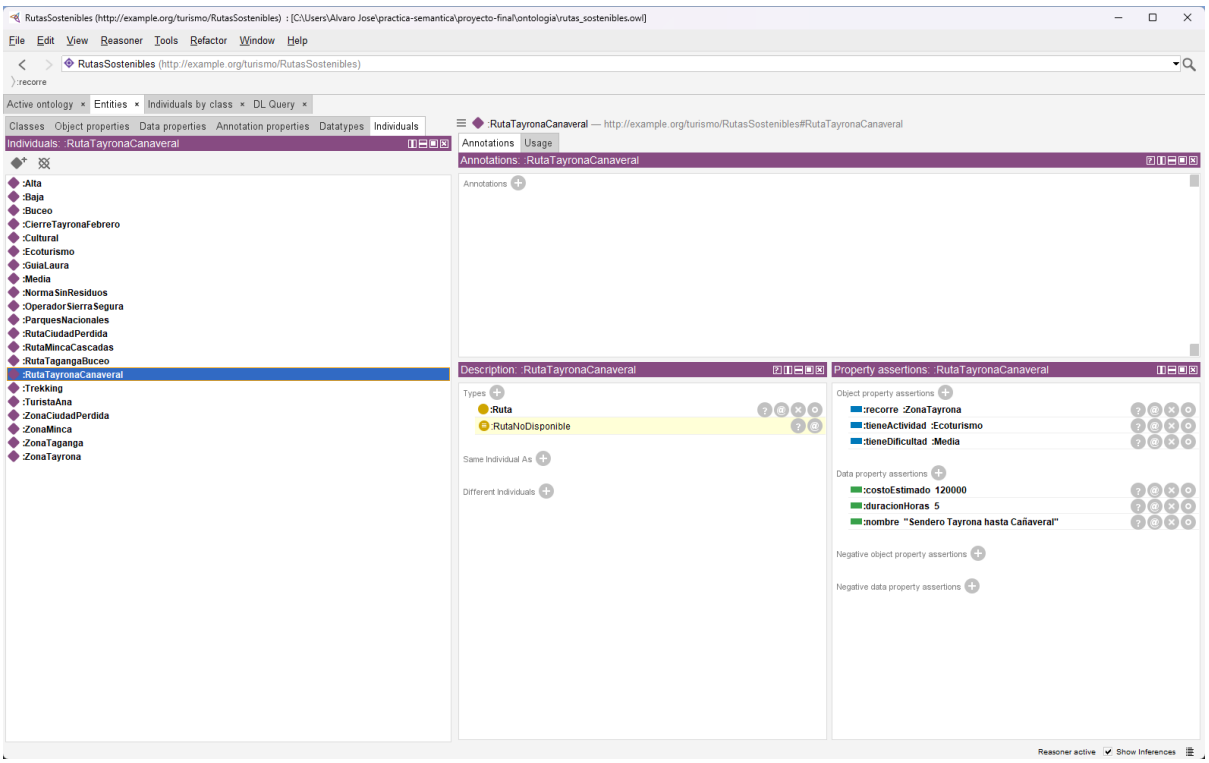


Figura 10. RutaTayronaCanaveral: el tipo RutaNoDisponible, en amarillo, fue inferido por HermiT

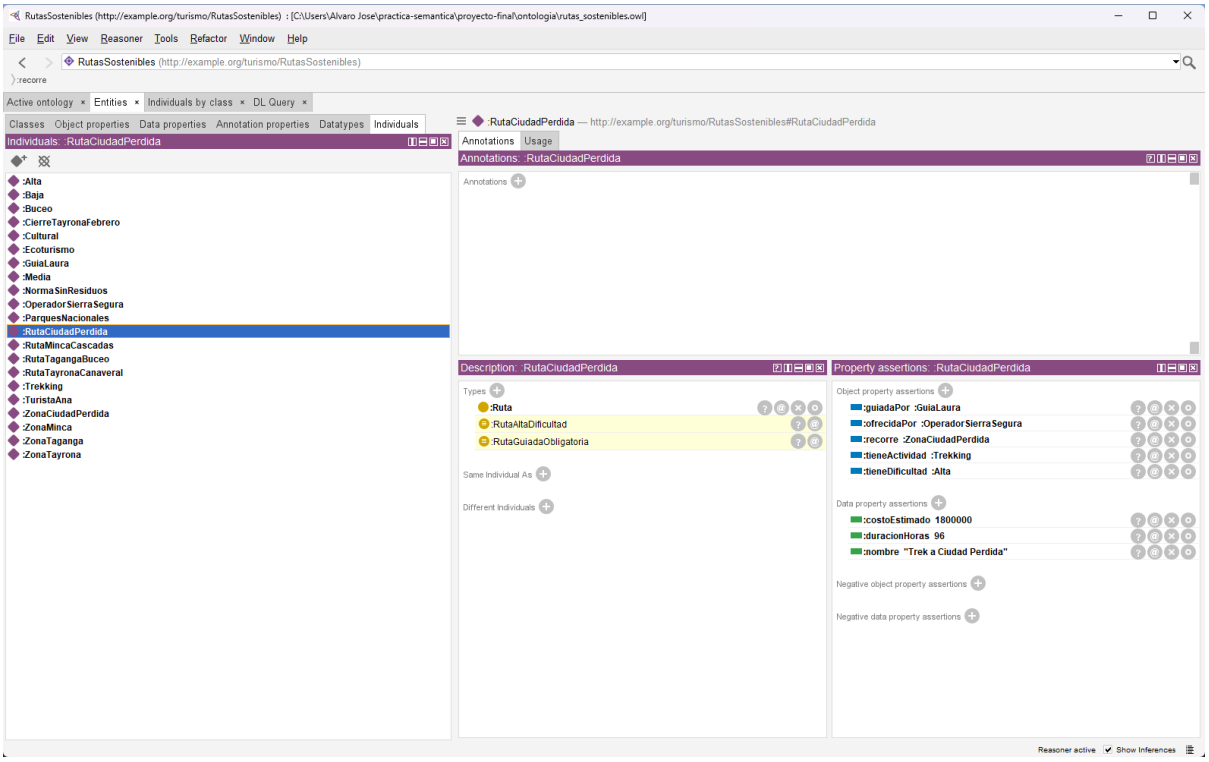


Figura 11. RutaCiudadPerdida: clasificada como RutaAltaDificultad y RutaGuiadaObligatoria

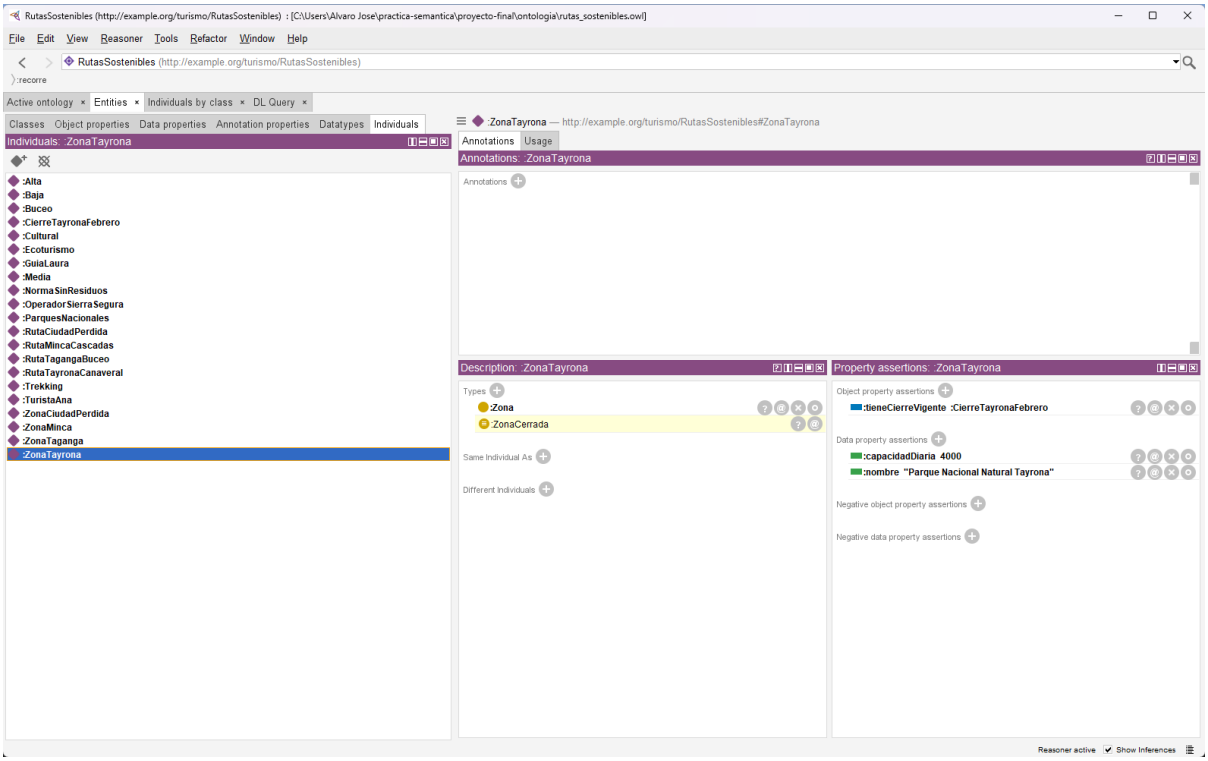


Figura 12. ZonaTayrona clasificada como ZonaCerrada

Sobre la tercera pregunta de competencia. Las preguntas 1 y 2 se responden dentro de Protégé, con las clases definidas. La 3 ("rutas aptas para esta turista") se responde con Cypher, y no es casualidad. OWL trabaja con el supuesto de mundo abierto: que una zona no tenga un cierre registrado no demuestra que esté abierta, así que el razonador no puede concluir "esta ruta es apta". Neo4j, en cambio, parte de que lo que no está en el grafo no existe, y por eso puede responder la pregunta. Esta división de trabajo es parte del diseño: la ontología define y valida, y el grafo consulta y recomienda.

8. Grafo de conocimiento

Archivo: `rutas_sostenibles.cypher`. Se ejecutó en Neo4j 5 y contiene 19 nodos y 19 relaciones.

Consulta	Pregunta	Resultado con fecha 2026-02-10
1	¿Qué rutas están disponibles en la fecha?	Taganga, Minca y Ciudad Perdida. El sendero del Tayrona queda fuera por el cierre
2	¿Quién ofrece y guía las rutas de trekking?	Trek a Ciudad Perdida, operador Sierra Segura, guía Laura
3	**Recomendación:** ¿qué ruta le sirve a Ana?	Cascadas y café en Minca

La consulta 3 es la que genera conocimiento no registrado: en el grafo no existe ninguna relación "recomendada". Se deduce combinando la dificultad que Ana soporta, su actividad preferida y el estado de la zona en esa fecha. Con una fecha fuera del cierre, el sendero del Tayrona también aparecería.

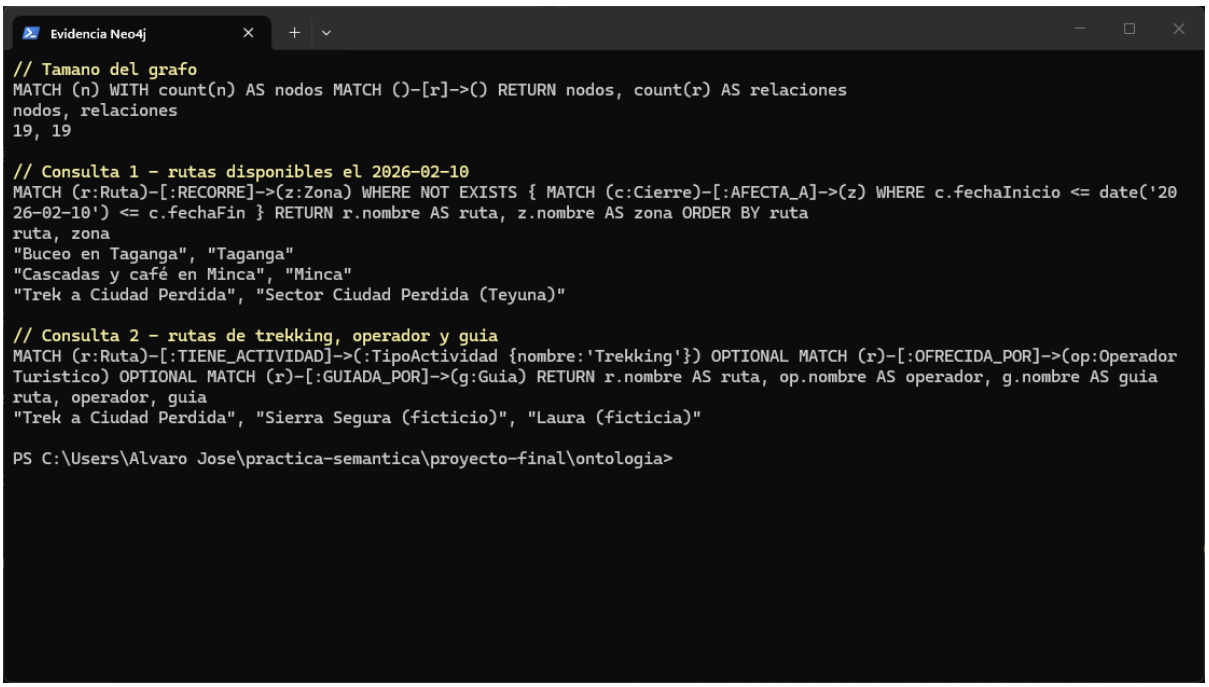


Figura 13. Tamaño del grafo y consultas 1 y 2 ejecutadas en Neo4j

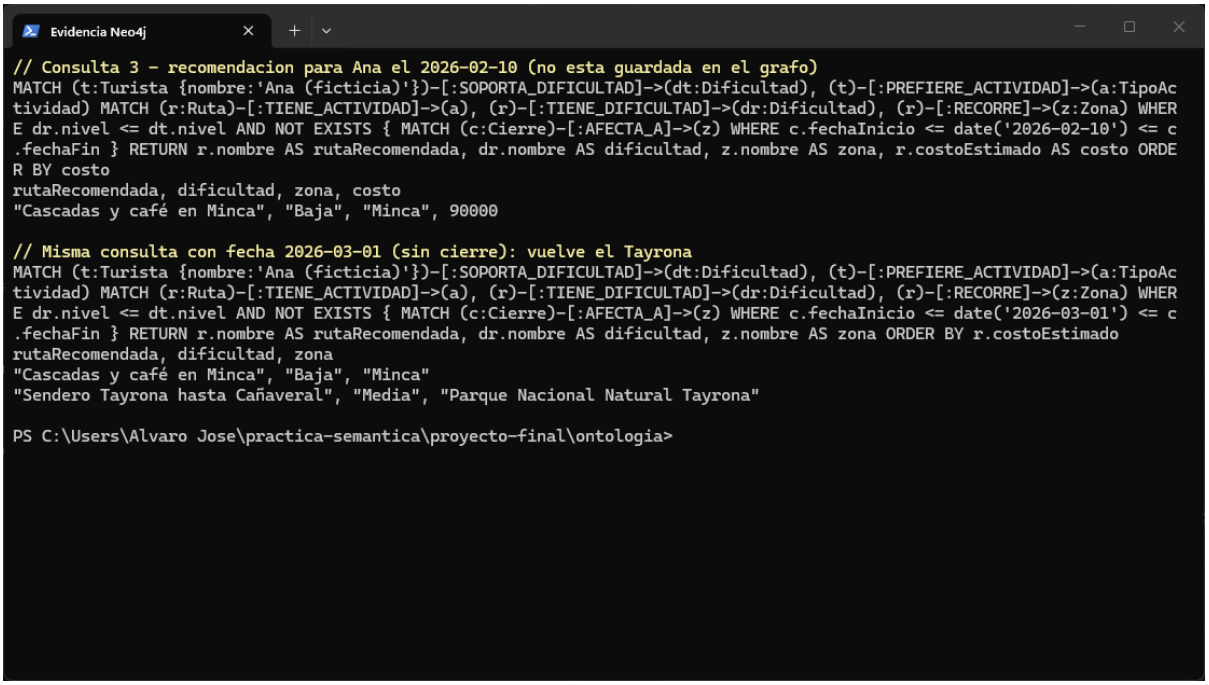


Figura 14. Consulta 3 (recomendación). Con la fecha del cierre el Tayrona no aparece; con una fecha sin cierre, vuelve a aparecer

9. Integración del diseño

- **Problema y funcionalidades:** cada funcionalidad responde a una necesidad de un perfil. Por ejemplo, "consultar disponibilidad" responde a que el turista no sabe si la zona está abierta.
- **Modelo conceptual y navegación:** cada clase relevante tiene una pantalla. `Ruta` tiene la lista y el detalle, `Cierre` tiene el panel de la autoridad y `Turista` tiene el perfil.
- **Ontología y dominio:** la ontología expresa las reglas (cierre, dificultad, guía obligatoria) como definiciones que el razonador puede aplicar.
- **Servicios y modelo:** cada servicio trabaja sobre un concepto del modelo (`/rutas`, `/zonas/{id}/cierres`, `/solicitudes`) y usa sus mismos nombres.

- **Grafo y conocimiento:** el grafo toma la estructura de la ontología y responde las consultas, incluida la recomendación.
- **Arquitectura:** la API une todo: recibe la petición del cliente, consulta el grafo y devuelve la respuesta explicada.

10. Entregables y repositorio

Entregable	Dónde está
Documento técnico consolidado	Este documento
Diagramas y modelos	Secciones 3, 5.2 y 5.3 de este documento
Prototipo navegable	`prototipo/index.html` (se abre en cualquier navegador). En línea: [pegar enlace de GitHub Pages]
Ontología en formato OWL	`ontologia/rutas_sostenibles.owl` (y su copia en Turtle)
Consultas Cypher	`grafo/rutas_sostenibles.cypher`
Evidencias de ejecución en Protégé y Neo4j	Carpeta `evidencias/` y figuras 7 a 14
Repositorio Git	Repositorio Git del equipo con su historial de commits: [pegar enlace de GitHub]
Presentación de la sustentación	`presentacion/Proyecto_Final_Rutas_Sostenibles.pptx`

Los datos de personas, operadores y cifras de capacidad son ficticios o sintéticos. La información sobre cierres del Parque Tayrona y sobre Ciudad Perdida viene de prensa y de sitios turísticos, y no está verificada con fuentes oficiales.

11. Referencias

Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Tesis doctoral, University of California, Irvine]. https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm

Hogan, A., Blomqvist, E., Cochez, M., d'Amato, C., de Melo, G., Gutierrez, C., Kirrane, S., Gayo, J. E. L., Navigli, R., Neumaier, S., Ngomo, A.-C. N., Polleres, A., Rashid, S. M., Rula, A., Schmelzeisen, L., Sequeda, J., Staab, S., & Zimmermann, A. (2021). Knowledge graphs. *ACM Computing Surveys*, *54*(4), 1–37. https://doi.org/10.1145/3447772

Koch, N., & Kraus, A. (2002). The expressive power of UML-based Web engineering. *Second International Workshop on Web-oriented Software Technology (IWWOST)*. https://www.pst.ifi.lmu.de/~kochn/IWWOST02-koch-kraus.PDF

Koch, N., Knapp, A., Zhang, G., & Baumeister, H. (2008). UML-based Web engineering: An approach based on standards. En G. Rossi, O. Pastor, D. Schwabe & L. Olsina (Eds.), *Web engineering: Modelling and implementing Web applications* (cap. 7). Springer. https://www.pst.ifi.lmu.de/People/former-members/koch/publications/2008/koch-et-al-uwe-springer-ch7.pdf

Nielsen, J. (2020). *10 usability heuristics for user interface design*. Nielsen Norman Group. https://www.nngroup.com/articles/ten-usability-heuristics/

Noy, N. F., & McGuinness, D. L. (2001). *Ontology development 101: A guide to creating your first ontology* (Stanford Knowledge Systems Laboratory Technical Report KSL-01-05). https://protege.stanford.edu/publications/ontology_development/ontology101-noy-mcguinness.html